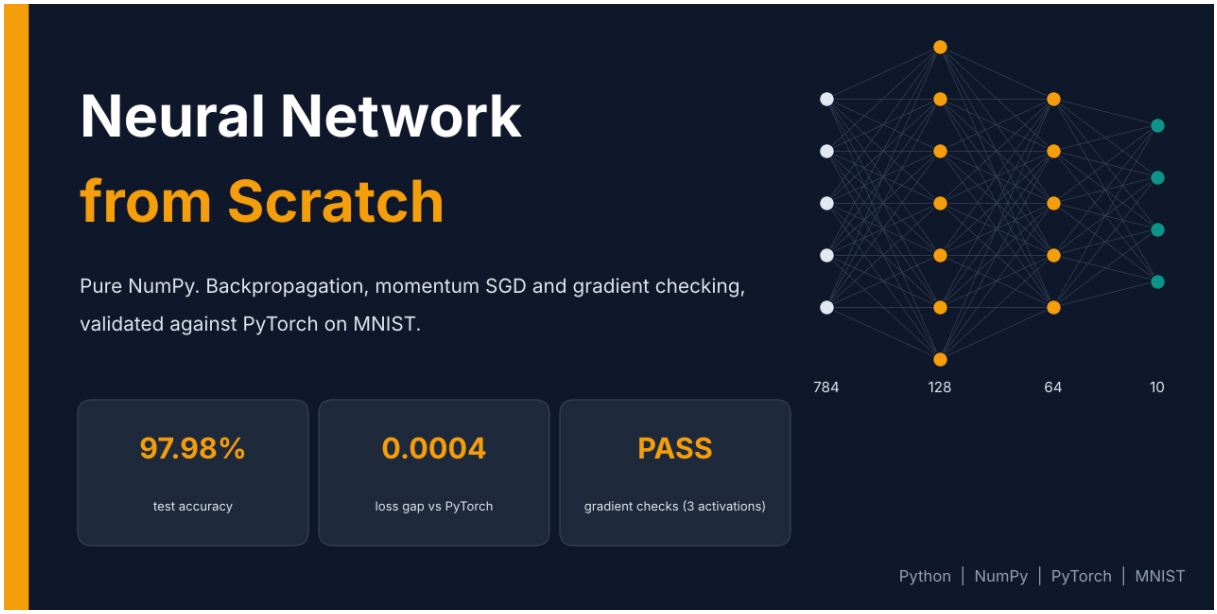
Neural Network from Scratch

A NumPy-only feedforward network trained on MNIST and validated against PyTorch

Project Report | Muhammad Faizan Farooq | November 2026



1. Objective

The goal of this project is to implement a complete feedforward neural network using only NumPy, with no machine learning framework inside the network code. The implementation covers forward propagation, backpropagation, several activation functions, mini-batch gradient descent and an optimizer with momentum. The network is trained on the MNIST handwritten digit dataset, its gradients are verified numerically with gradient checking, and its results are compared with an equivalent PyTorch model.

2. Summary of Results

Metric	Result
NumPy test accuracy	97.98%
PyTorch test accuracy	97.98%
Test loss (NumPy / PyTorch)	0.0752 / 0.0748
Max validation-loss gap over 20 epochs	6.75e-03
Gradient check (ReLU, Tanh, Sigmoid)	All gradients correct

3. System Architecture

The project is organised so that the network itself (the **nn/** package) imports only NumPy. Matplotlib, PyTorch and torchvision are used only for plotting, dataset download and the comparison model. The figure below shows the data flow, the forward and backward passes, and the supporting components.

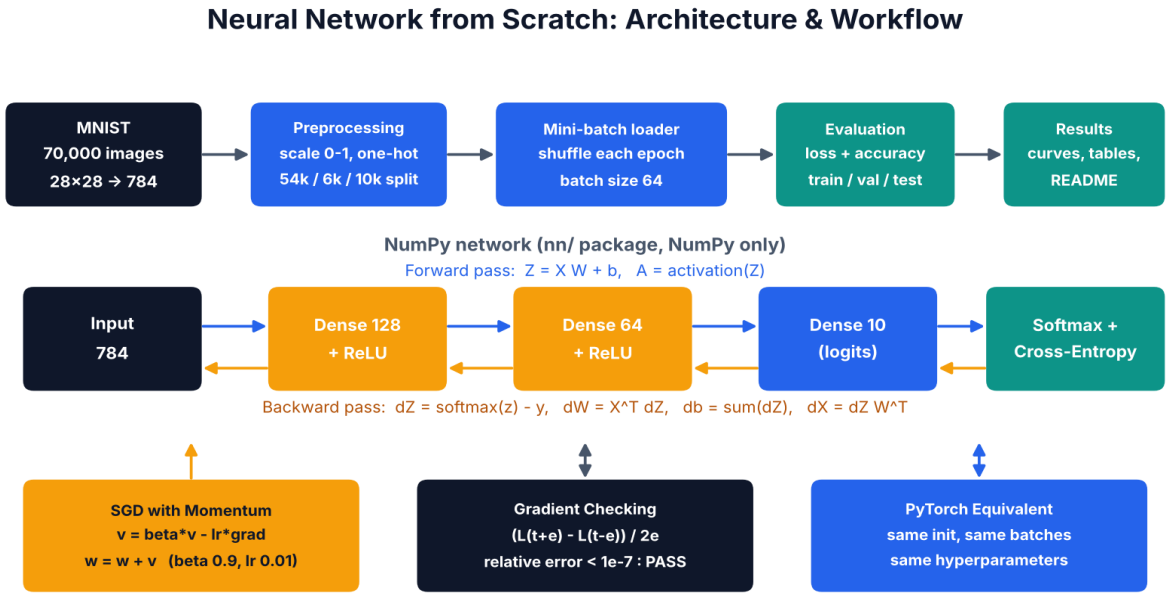


Figure 1: Architecture and workflow of the project.

4. Implementation

4.1 Data

MNIST images (28x28) are flattened to 784 features and scaled to the range [0, 1]. The 60,000 training images are split 90/10 into 54,000 training and 6,000 validation samples. The 10,000 test images are used only for the final evaluation. Labels are one-hot encoded.

4.2 Dense layer and initialization

Each layer computes Z = XW + b and caches its input X for the backward pass. The gradients are dW = X^T dZ, db = sum(dZ) and dX = dZ W^T. Weights use **He initialization** (variance $2/n_{in}$) for ReLU layers and **Xavier/Glorot initialization** for Tanh, Sigmoid and the output layer, which keeps the variance of activations and gradients stable across layers.

4.3 Activation functions

Activation	Forward	Derivative
ReLU	max(0, z)	1 if z > 0, else 0
Sigmoid	1 / (1 + e^-z)	a (1 - a)
Tanh	tanh(z)	1 - a^2
Softmax	e^zi / sum(e^zj)	Jacobian-vector product

4.4 Loss and backpropagation

Softmax and cross-entropy are fused into one loss. The combined gradient with respect to the logits is simply **(softmax(z) - y) / N**, and the log-sum-exp trick avoids overflow and log(0). Backpropagation applies the chain rule from the output towards the input: each activation multiplies by its local derivative and each dense layer produces dW, db and dX.

4.5 Mini-batch gradient descent and momentum

Every epoch the training set is shuffled and processed in batches of 64. Updates use SGD with momentum:

```
v = beta * v - lr * grad
w = w + v   (beta = 0.9, lr = 0.01)
```

Momentum keeps a decaying average of past gradients, which damps oscillations and speeds up convergence. Setting beta = 0 gives plain SGD, which is used in the ablation study.

5. Gradient Checking

Backpropagation bugs are silent: the model still trains, only worse. To rule them out, every analytic gradient is compared with a centered finite difference:

```
(L(theta + e) - L(theta - e)) / (2e),  e = 1e-5
relative error = ||g_analytic - g_numeric|| / (||g_analytic|| + ||g_numeric||)
```

The check runs on a small 5-4-4-3 network in float64 for ReLU, Tanh and Sigmoid. A relative error below 1e-7 counts as a pass. Biases are set to small random values for the test, because a ReLU pre-activation of exactly zero is a non-differentiable point. **Result: all parameters of all three activations passed** (output: ALL GRADIENTS CORRECT).

6. Experimental Setup

Parameter	Value
Architecture	784 - 128 - 64 - 10
Hidden activation	ReLU (Softmax inside the loss)
Optimizer	SGD, lr = 0.01, momentum = 0.9
Batch size / epochs	64 / 20
Split	54,000 train / 6,000 val / 10,000 test
Seed	42

For a fair comparison, the PyTorch model uses the same architecture, the same initial weights (copied from the NumPy model), the same mini-batch order and the same hyperparameters.

7. Results

7.1 NumPy vs PyTorch

Metric	NumPy	PyTorch	Abs. difference
Final train loss	0.0045	0.0045	0.0000
Final val loss	0.1041	0.1040	0.0001
Final val accuracy	0.9757	0.9762	0.0005
Test loss	0.0752	0.0748	0.0004
Test accuracy	0.9798	0.9798	0.0000
Training time (s)	55.6	52.1	3.5

The two implementations reach the same test accuracy, and the maximum validation-loss difference over all epochs is 6.75e-03. The small remaining gap comes from floating-point details. NumPy was about 3.5 seconds slower over 20 epochs.

7.2 Ablation study (10 epochs)

Configuration	Val acc	Test acc	Test loss
ReLU + momentum 0.9	0.9745	0.9784	0.0716
ReLU + plain SGD	0.9448	0.9494	0.1741
Tanh + momentum 0.9	0.9755	0.9747	0.0819
Sigmoid + momentum 0.9	0.9350	0.9418	0.1983

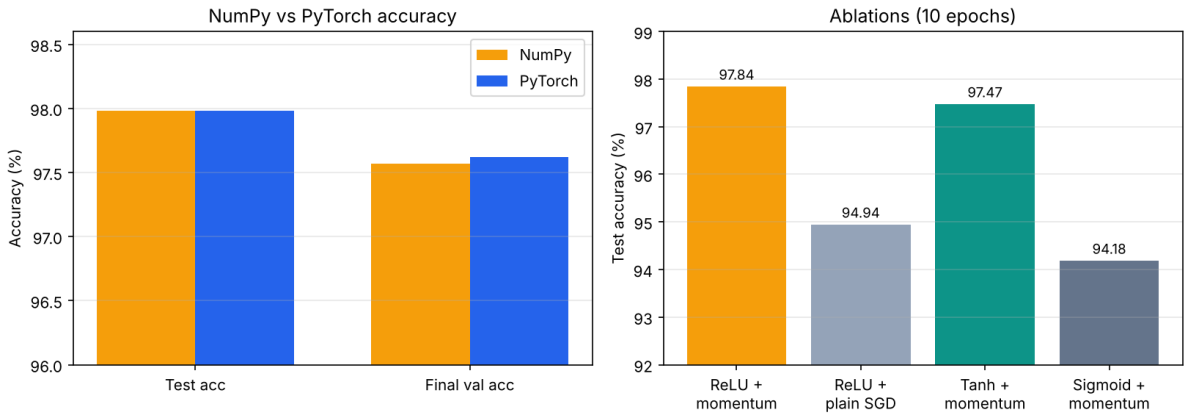


Figure 2: Accuracy comparison (left) and ablation results (right).

8. Conclusions

- **Correctness:** gradient checking passes for ReLU, Tanh and Sigmoid, so the hand-written backpropagation is correct.
- **Match with PyTorch:** the from-scratch model reaches the same 97.98% test accuracy as PyTorch, with a test-loss difference of only 0.0004.
- **Momentum:** with momentum 0.9, ReLU reached 97.84% test accuracy after 10 epochs versus 94.94% with plain SGD at the same learning rate.

- **Activations:** ReLU and Tanh trained well; ReLU gave the best test accuracy and loss. Sigmoid was clearly slower because it saturates and produces small gradients.
- **Overfitting:** training accuracy reaches about 99.96% while validation accuracy stays near 97.6%, and validation loss stops improving after about epoch 10. Dropout, weight decay or early stopping are natural next steps.

9. How to Run

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python download_data.py
python gradient_check.py
python train_numpy.py
python train_pytorch.py
python compare.py
python experiments.py
```